

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN BỐN
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã hoàn thành quá trình huấn luyện mô hình YOLOv8n trong tuần thứ ba, tuần thứ tư của dự án tập trung vào việc nâng cao hiệu suất mô hình và triển khai hệ thống dưới dạng API.

Các mục tiêu chính trong tuần này bao gồm:

- Huấn luyện mô hình YOLOv8s để cải thiện độ chính xác
- So sánh hiệu suất giữa YOLOv8n (tuần 3) và YOLOv8s
- Đánh giá mô hình trên cả tập validation và test
- Phân tích khả năng tổng quát hóa của mô hình
- Xây dựng API dự đoán ảnh sử dụng FastAPI
- Tích hợp mô hình vào hệ thống backend

Việc hoàn thành các mục tiêu này giúp chuyển đổi từ giai đoạn nghiên cứu mô hình sang triển khai ứng dụng thực tế.

2. Huấn luyện và đánh giá mô hình YOLOv8s

2.1. Huấn luyện mô hình YOLOv8s

Sau khi nhận thấy mô hình YOLOv8n vẫn còn hạn chế, đặc biệt ở lớp “Without Helmet”, trong tuần này em tiến hành huấn luyện mô hình YOLOv8s nhằm cải thiện khả năng học đặc trưng.

Lệnh thực hiện:

```
yolo detect train data=dataset/data.yaml model=yolov8s.pt epochs=100  
imgsz=416 batch=8 name=helmet_s
```

Giải thích các tham số:

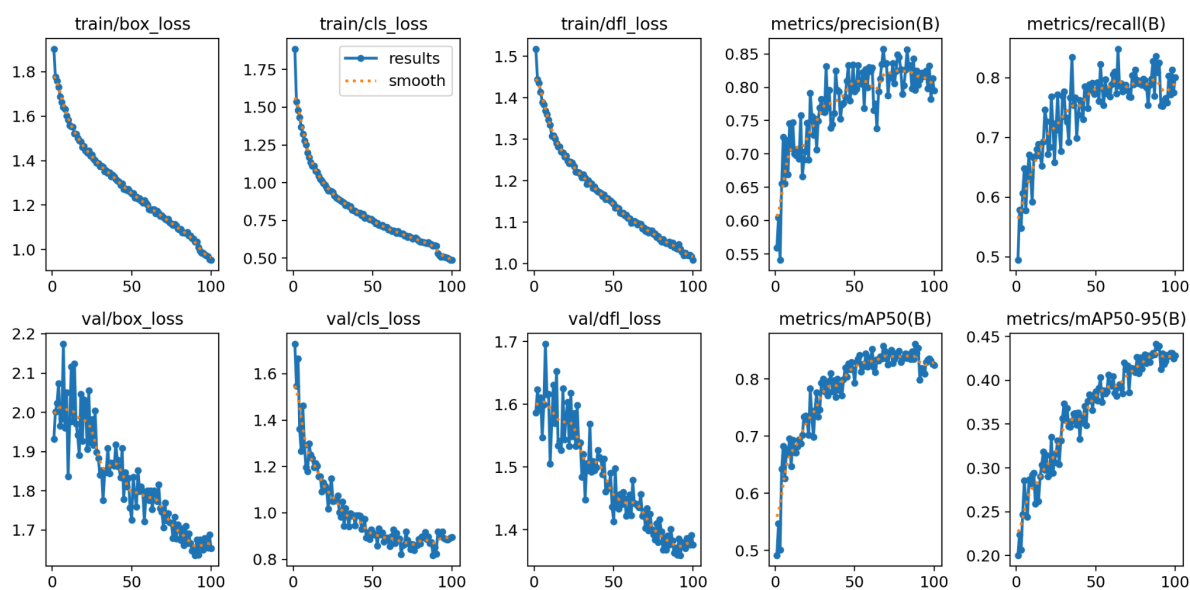
- **data=dataset/data.yaml:** chỉ định file cấu hình dataset, bao gồm đường dẫn tới các tập train, validation và test
- **model=yolov8s.pt:** sử dụng mô hình YOLOv8 phiên bản small đã được pretrained
- **epochs=100:** số epoch nhỏ để kiểm tra nhanh quá trình huấn luyện
- **imgsz=416:** kích thước ảnh đầu vào, phù hợp với dataset đã được resize trước đó
- **batch=8:** số lượng ảnh được xử lý trong mỗi batch, phù hợp với tài nguyên GPU hiện có

- **name=helmet_s**: tên thư mục lưu kết quả huấn luyện

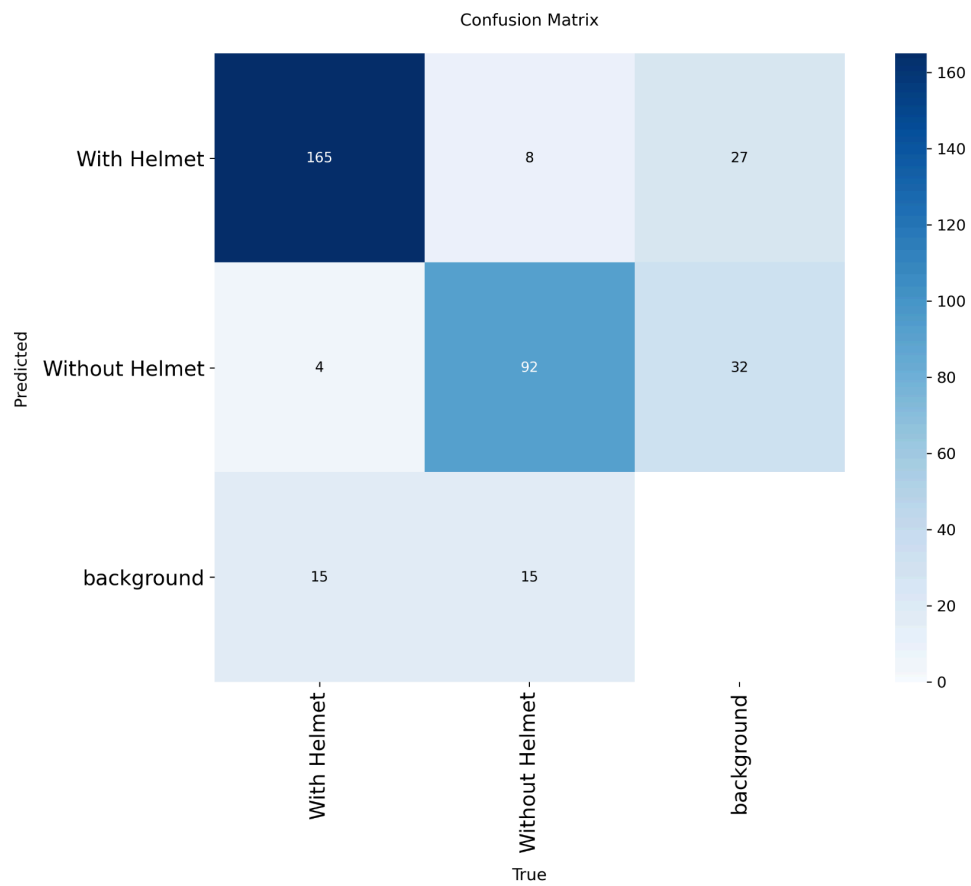
Sau khi thực hiện lệnh trên, hệ thống tiến hành huấn luyện mô hình trong 2 epoch và lưu kết quả tại thư mục: **runs/detect/helmet_s/**

Thư mục này chứa các thông tin quan trọng phục vụ cho việc phân tích mô hình, bao gồm:

- File trọng số tốt nhất: best.pt
- File trọng số cuối cùng: last.pt
- Biểu đồ loss và metric: results.png



- Ma trận nhầm lẫn (confusion matrix)



- Các hình ảnh minh họa kết quả dự đoán



2.2. Đánh giá trên tập Validation

Để kiểm chứng khách quan, em tiến hành chạy lại công cụ Validation trên trọng số tốt nhất (best.pt) vừa thu được

yolo detect val model=runs/detect/helmet_s/weights/best.pt

data=dataset/data.yaml name=val-100-s

Mô hình đạt được các chỉ số đánh giá trên tập validation như sau:

Chỉ số	Giá trị
Precision	0.796
Recall	0.837
mAP@0.5	0.862

mAP@0.5:0.95	0.442
--------------	-------

Kết quả theo từng lớp đối tượng:

Lớp	Precision	Recall	mAP@0.5	mAP@0.5:0.95
With Helmet	0.84	0.88	0.902	0.486
Without Helmet	0.752	0.793	0.821	0.397

Tốc độ xử lý trung bình 1 ảnh (ms):

preprocess	inference	postprocess	loss	Tổng
2.2	14.1	6.2	0.0	22.5

So sánh với huấn luyện mô hình nano:

Chỉ số	small	nano
Precision	0.796	0.776
Recall	0.837	0.805
mAP@0.5	0.862	0.826
mAP@0.5:0.95	0.442	0.389

Nhận xét:

Thông qua việc đánh giá trên tập validation và so sánh trực tiếp với mô hình YOLOv8n (Nano) đã huấn luyện trước đó, có thể rút ra những nhận xét chuyên sâu về hiệu suất của mô hình YOLOv8s (Small) như sau:

- Sự cải thiện toàn diện nhờ gia tăng độ sâu mạng:
 - Tất cả các chỉ số đánh giá cốt lõi đều ghi nhận sự tăng trưởng rõ rệt. Cụ thể, mAP@0.5 tăng từ 0.826 lên 0.862, và Recall tổng thể tăng từ 0.805 lên 0.837. Việc chuyển từ kiến trúc Nano sang Small đồng nghĩa với việc gia tăng số lượng tham số (parameters) và độ sâu của mạng nơ-ron. Điều này giúp mô hình có khả năng trích xuất và học được các đặc trưng phức tạp hơn, từ đó đưa ra các dự đoán chính xác hơn so với bản rút gọn.

- Khắc phục hiệu quả điểm yếu ở lớp thiếu số (Without Helmet):
 - Đây là điểm sáng lớn nhất của phiên bản YOLOv8s. Ở mô hình Nano, lớp "Without Helmet" gặp khó khăn do thiếu hụt dữ liệu (Recall chỉ đạt 0.713). Tuy nhiên, trên mô hình Small, chỉ số Recall của lớp này đã bứt phá lên mức 0.793, và mAP@0.5 đạt 0.821. Điều này chứng minh rằng một mạng nơ-ron có dung lượng lớn hơn sẽ có khả năng kháng lại sự mất cân bằng dữ liệu tốt hơn, giảm thiểu đáng kể hiện tượng bỏ sót những người vi phạm không đội mũ bảo hiểm.
- Chất lượng định vị khung hình (Bounding Box) sắc nét hơn:
 - Chỉ số mAP@0.5:0.95 đo lường mức độ khớp (IoU) khắt khe giữa khung hình dự đoán và thực tế. Chỉ số này đã có sự nhảy vọt từ 0.389 (Nano) lên 0.442 (Small). Điều này cho thấy mô hình không chỉ nhận diện đúng đối tượng mà còn vẽ khung bounding box "ôm" sát vào đầu người/mũ bảo hiểm hơn, hạn chế các khoảng trống thừa.
- Sự đánh đổi hợp lý giữa Độ chính xác và Tốc độ:
 - Đổi lại sự gia tăng về độ chính xác, tốc độ xử lý của mô hình Small chậm hơn so với bản Nano. Tổng thời gian xử lý (bao gồm tiền xử lý, suy luận và hậu xử lý) tăng từ 9.5 ms lên 22.5 ms/ảnh . Tuy nhiên, nếu quy đổi, tốc độ này tương đương với khoảng 44 FPS (khung hình/giây). Trong kỹ thuật thị giác máy tính, mức >30 FPS đã được xếp vào tiêu chuẩn xử lý thời gian thực (Real-time). Do đó, sự suy giảm tốc độ này là hoàn toàn chấp nhận được và không gây ảnh hưởng đến trải nghiệm người dùng trong môi trường triển khai thực tế.

Kết luận:

Việc thử nghiệm và nâng cấp lên mô hình YOLOv8s là một quyết định kỹ thuật hoàn toàn đúng đắn. Mô hình đã chứng minh được sự vượt trội về khả năng tổng quát hóa, đặc biệt là trong việc nhận diện chính xác lớp đối tượng khó ("Without Helmet") và vẽ khung hình chuẩn xác hơn. Dù tài nguyên tính toán yêu cầu cao hơn đôi chút, tốc độ 44 FPS vẫn đảm bảo hiệu năng xuất sắc cho các hệ thống giám sát video trực tiếp.

2.3. Đánh giá và chốt mô hình trên tập Test

Mặc dù tập Validation cho thấy mô hình YOLOv8s vượt trội hơn YOLOv8n, nhưng để đảm bảo tính khách quan tuyệt đối và kiểm chứng khả năng hoạt động trên dữ liệu thực tế (chưa từng xuất hiện trong bất kỳ giai đoạn train hay validation nào), em tiến hành đánh giá lại cả hai trọng số tốt nhất (best.pt) trên tập Test. Đây là bước quyết định để chốt mô hình cuối cùng phục vụ cho việc triển khai ứng dụng.

Lệnh thực hiện:

- Đánh giá YOLOv8n trên tập test

```
yolo detect val model=runs/detect/helmet_final/weights/best.pt  
data=dataset/data.yaml split=test name=test-100-n
```

- Đánh giá YOLOv8s trên tập test

```
yolo detect val model=runs/detect/helmet_s/weights/best.pt  
data=dataset/data.yaml split=test name=test-100-s
```

Bảng so sánh:

Chỉ số	small	nano	Độ chênh lệch
Precision	0.896	0.849	+0.047
Recall	0.828	0.763	+0.065
mAP@0.5	0.889	0.842	+0.047
mAP@0.5:0.95	0.459	0.421	+0.038

Phân tích hiệu suất chuyên sâu đối với lớp "Without Helmet"

Chỉ số	small	nano	Độ chênh lệch
Precision	0.856	0.797	+0.059
Recall	0.803	0.656	+0.147
mAP@0.5	0.835	0.766	+0.069
mAP@0.5:0.95	0.407	0.364	+0.043

Từ kết quả trên, có thể nhận thấy:

- Khả năng tổng quát hóa (Generalization) xuất sắc:

- Các chỉ số mAP và Recall của cả hai mô hình trên tập Test đều duy trì ở mức cao và bám sát với kết quả trên tập Validation. Điều này là minh chứng rõ nét cho việc cấu hình siêu tham số (Hyperparameters) và chiến lược huấn luyện (Early Stopping) đã hoạt động hiệu quả, giúp mô hình nắm bắt được đặc trưng thực sự của dữ liệu mà không hề bị mắc lỗi học vẹt (Overfitting).
- Sự bứt phá mang tính quyết định ở lớp thiểu số (Minority Class):
 - Như đã phân tích ở các tuần trước, bài toán đối mặt với sự mất cân bằng dữ liệu khi lớp "Without Helmet" có số lượng mẫu ít hơn và đặc trưng hình ảnh khó phân biệt hơn.
 - Trên bản YOLOv8n, chỉ số Recall của lớp này chỉ đạt 0.656 (bỏ sót khoảng 34% trường hợp vi phạm). Tuy nhiên, khi chuyển sang kiến trúc YOLOv8s, chỉ số này đã tăng vọt lên 0.803 (giảm tỷ lệ bỏ sót xuống chỉ còn dưới 20%). Sự gia tăng dung lượng mạng (từ 3 triệu lên 11.1 triệu tham số) đã phát huy tối đa tác dụng trong việc định vị và phân loại các đối tượng khó.
- Hiệu năng xử lý thực tế cực kỳ tối ưu:
 - Một phát hiện thú vị trên tập Test là cả hai mô hình đều cho tốc độ suy luận (Inference time) gần như tương đương nhau trên GPU RTX 3050 Ti: ~8.7ms đối với Nano và ~8.6ms đối với Small.
 - Tổng thời gian cho toàn bộ một chu trình (Preprocess + Inference + Postprocess) của bản Small chỉ mất khoảng 11.6 ms/ảnh. Tốc độ này tương đương với ~86 FPS, vượt xa mức cấu hình tối thiểu của hệ thống thời gian thực. Điều này hoàn toàn xóa bỏ lo ngại về việc mô hình Small sẽ làm chậm hệ thống.

Kết luận chốt mô hình:

Dựa trên các thực nghiệm đối chứng toàn diện, trọng số best.pt của mô hình YOLOv8s chính thức được lựa chọn làm mô hình cuối cùng để triển khai. Sự kết hợp hoàn hảo giữa độ chính xác cao (mAP@0.5 đạt 0.889), khả năng khắc phục hiện tượng bỏ sót vi phạm (Recall "Without Helmet" đạt 0.803) và tốc độ xử lý nhanh (~86 FPS)

biến đây trở thành một lõi AI lý tưởng, hoàn toàn sẵn sàng để tích hợp vào hệ thống Web API phục vụ giám sát giao thông thực tế.

Tuy nhiên em vẫn sẽ giữ lại mô hình nano để phục vụ cho những tình huống cụ thể cần tốc độ xử lý nhanh hơn, khi mô hình small không đủ đáp ứng phục vụ công việc.

3. Xây dựng web dự đoán bằng FastAPI

3.1. Mục tiêu

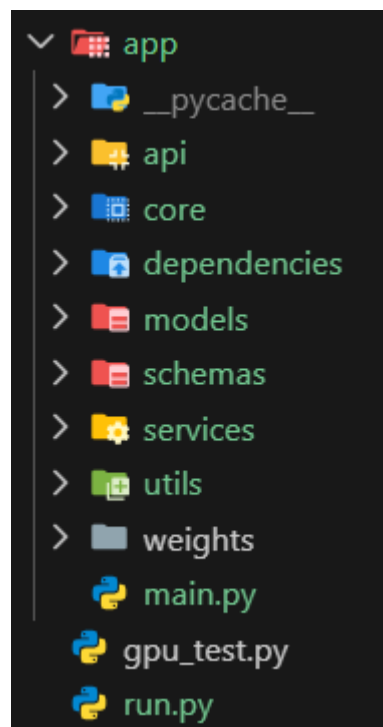
Trong tuần này, em bước đầu triển khai hệ thống dưới dạng web service nhằm kiểm tra khả năng tích hợp mô hình vào ứng dụng thực tế.

Tuy nhiên, phạm vi chỉ dừng lại ở mức xây dựng một API đơn giản cho phép gửi ảnh lên và nhận kết quả dự đoán từ mô hình.

Hệ thống hiện tại được xem như một bản demo ban đầu, chưa phải phiên bản hoàn chỉnh cuối cùng.

4.2. Cấu trúc hệ thống

Dự án được tổ chức theo kiến trúc module:



4.3. Tổng quan hệ thống

Các chức năng đã triển khai:

- Khởi tạo server bằng FastAPI
- Load mô hình YOLO khi hệ thống khởi động
- Xây dựng endpoint /helmet/predict
- Nhận ảnh từ người dùng
- Thực hiện inference bằng mô hình
- Trả về kết quả dưới dạng JSON

Các chức năng chưa thực hiện:

- Chưa có giao diện người dùng (frontend)
- Chưa xử lý video hoặc realtime
- Chưa tối ưu hiệu năng (batch, queue, async nâng cao)
- Chưa triển khai production (Docker, deploy server)

4.4. Cơ chế hoạt động

Quy trình xử lý của hệ thống:

Client → Upload Image → FastAPI → YOLO Model → Return Result

Chi tiết:

1. Người dùng gửi ảnh qua API
2. Server đọc và chuyển đổi ảnh sang định dạng OpenCV
3. Mô hình YOLO thực hiện dự đoán
4. Kết quả được xử lý:
5. Trích xuất bounding box
6. Lấy confidence
7. Ảnh sau khi detect được encode base64
8. Trả về JSON cho client

4.5. Load mô hình

Mô hình được load một lần khi khởi động:

```
app.state.model = YOLO(setting.MODEL_PATH)
```

Mục đích:

- Tránh load lại model nhiều lần
- Giảm thời gian xử lý mỗi request

4.6. Endpoint dự đoán

POST /helmet/predict

Input: File ảnh

Output:

```
{
  "detections": [
    {
      "class_id": 0,
      "confidence": 0,
      "bbox": {
        "x1": 0,
        "y1": 0,
        "x2": 0,
        "y2": 0
      }
    }
  ],
  "total_detections": 0,
  "image_base64": "string"
}
```

4.7. Đánh giá mức độ hoàn thiện

Hệ thống hiện tại:

- Hoạt động đúng chức năng cơ bản
- Có thể dự đoán ảnh từ client
- Trả về kết quả trực quan

Tuy nhiên:

- Mới ở mức thử nghiệm
- Chưa tối ưu cho hệ thống thực tế
- Chưa đảm bảo hiệu năng khi nhiều request

4.8. Nhận xét

Việc xây dựng API trong tuần này chủ yếu nhằm:

- Kiểm chứng khả năng tích hợp mô hình vào backend
- Làm nền tảng cho các bước phát triển tiếp theo
- Chuẩn bị cho việc xây dựng hệ thống hoàn chỉnh

4. Kết quả đạt được trong tuần 4

Sau khi hoàn thành các công việc trong tuần thứ bốn, các kết quả chính đạt được bao gồm:

- Huấn luyện thành công mô hình YOLOv8s
 - Cải thiện hiệu suất so với YOLOv8n
 - Đánh giá mô hình trên cả validation và test
 - Xác nhận mô hình không bị overfitting
- Xây dựng thành công API bằng FastAPI
 - Triển khai pipeline inference hoàn chỉnh
 - Chuẩn bị nền tảng cho hệ thống thực tế

Những kết quả đạt được trong tuần thứ bốn cho thấy mô hình đã được huấn luyện thành công, đạt hiệu suất tốt và có khả năng hoạt động trong một môi trường ứng dụng thực tế. Từ làm nền tảng cho việc phát triển hệ thống hoàn chỉnh.

5. Khó khăn gặp phải và cách khắc phục

Trong quá trình thực hiện tuần thứ tư của dự án, một số khó khăn đã phát sinh liên quan đến việc so sánh mô hình, đánh giá khả năng tổng quát hóa, và bước đầu triển khai hệ thống dưới dạng API. Các vấn đề này chủ yếu xuất phát từ việc đảm bảo tính chính xác của kết quả thực nghiệm cũng như tích hợp mô hình vào môi trường ứng dụng thực tế.

5.1. Khó khăn trong việc so sánh hai mô hình YOLOv8n và YOLOv8s

Một trong những khó khăn quan trọng trong tuần này là việc đánh giá và so sánh chính xác hiệu suất giữa hai mô hình YOLOv8n và YOLOv8s.

Do hai mô hình có kích thước và số lượng tham số khác nhau, việc chỉ dựa vào một chỉ số đơn lẻ (ví dụ mAP@0.5) có thể không phản ánh đầy đủ hiệu suất thực tế.

Ngoài ra, sự khác biệt giữa các chỉ số trên tập validation và tập test cũng gây khó khăn trong việc đưa ra kết luận chính xác.

Để khắc phục vấn đề này, em đã:

- Đánh giá mô hình trên cả hai tập validation và test
 - So sánh đồng thời nhiều chỉ số:
 - Precision
 - Recall
 - mAP@0.5
 - mAP@0.5:0.95
 - Phân tích riêng theo từng lớp đối tượng

Nhờ đó, việc so sánh trở nên toàn diện hơn, giúp xác định rõ ràng rằng mô hình YOLOv8s có hiệu suất tốt hơn so với YOLOv8n.

5.2. Khó khăn trong việc đánh giá khả năng overfitting của mô hình

Một vấn đề khác trong tuần này là việc xác định liệu mô hình có bị overfitting hay không, đặc biệt khi kích thước của tập validation và test còn hạn chế.

Khi số lượng dữ liệu đánh giá không lớn, các chỉ số như mAP có thể dao động và không phản ánh chính xác khả năng tổng quát hóa của mô hình. Điều này gây khó khăn trong việc kết luận mô hình có học tốt hay chỉ đang “ghi nhớ” dữ liệu.

Để giải quyết vấn đề này, em đã:

- So sánh kết quả giữa tập validation và test
- Quan sát sự chênh lệch giữa các chỉ số:
 - Nếu test thấp hơn nhiều => có thể overfitting
 - Nếu tương đương => mô hình tổng quát tốt
- Phân tích thêm kết quả dự đoán thực tế

Kết quả cho thấy các chỉ số trên tập test tương đương với trên tập validation, từ đó cho thấy mô hình không có dấu hiệu overfitting rõ rệt.

5.3. Khó khăn trong việc tích hợp mô hình vào FastAPI

Trong quá trình xây dựng API, một khó khăn đáng chú ý là việc tích hợp mô hình YOLO vào hệ thống FastAPI sao cho hiệu quả.

Nếu không xử lý đúng, mô hình có thể bị load lại nhiều lần cho mỗi request, gây tốn tài nguyên và làm chậm hệ thống.

Để giải quyết vấn đề này, em đã:

- Sử dụng cơ chế lifespan của FastAPI để load model khi khởi động
- Lưu model vào app.state để tái sử dụng
- Sử dụng Dependency Injection để truyền model vào endpoint

Cách tiếp cận này giúp:

- Giảm thời gian xử lý
- Tránh lãng phí tài nguyên
- Đảm bảo hệ thống hoạt động ổn định

5.4. Khó khăn trong việc thiết kế API ở mức tối thiểu nhưng vẫn hợp lý

Trong tuần này, mục tiêu chỉ là xây dựng một API ở mức demo, tuy nhiên vẫn cần đảm bảo thiết kế đủ rõ ràng để có thể mở rộng trong tương lai.

Khó khăn đặt ra là:

- Nếu thiết kế quá đơn giản => khó mở rộng
- Nếu thiết kế quá phức tạp => mất nhiều thời gian

Để cân bằng, em đã:

- Tổ chức code theo dạng module
- Tách biệt rõ logic xử lý và định nghĩa dữ liệu
- Chỉ triển khai các chức năng cần thiết nhất

Cách tiếp cận này giúp hệ thống:

- Đơn giản nhưng vẫn có cấu trúc rõ ràng
- Dễ dàng nâng cấp trong các tuần tiếp theo

5. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã hoàn thành việc huấn luyện mô hình YOLOv8s và bước đầu triển khai API dự đoán trong tuần thứ tư, tuần tiếp theo của dự án sẽ tập trung vào việc hoàn thiện hệ thống và mở rộng khả năng ứng dụng thực tế. Các công việc này nhằm nâng cao tính ổn định, hiệu năng cũng như khả năng triển khai của hệ thống phát hiện mũ bảo hiểm.

6.1. Hoàn thiện và tối ưu API

Mặc dù API đã được xây dựng ở mức cơ bản trong tuần 4, hệ thống hiện tại mới chỉ dừng ở mức demo và chưa được tối ưu cho môi trường thực tế. Do đó, trong tuần tiếp theo, em sẽ tiến hành cải thiện API thông qua các hướng sau:

Tối ưu quá trình xử lý ảnh:

- Giảm thời gian decode và encode ảnh
- Tối ưu pipeline inference

Cải thiện cấu trúc code:

- Tách biệt rõ hơn giữa các module
- Chuẩn hóa response và xử lý lỗi

Bổ sung các chức năng cần thiết:

- Logging request và response
- Xử lý ngoại lệ (exception handling)

Việc tối ưu này giúp hệ thống hoạt động ổn định hơn và sẵn sàng cho việc mở rộng trong tương lai.

6.2. Mở rộng chức năng dự đoán

Hiện tại hệ thống chỉ hỗ trợ dự đoán trên ảnh tĩnh. Trong tuần tiếp theo, em sẽ mở rộng khả năng của hệ thống để xử lý các dạng dữ liệu khác:

- Dự đoán trên video:
 - Đọc video frame-by-frame
 - Áp dụng mô hình YOLO cho từng frame
- Hỗ trợ realtime (camera):
 - Sử dụng webcam để nhận dữ liệu trực tiếp
 - Hiển thị kết quả detection theo thời gian thực

Việc mở rộng này giúp hệ thống tiến gần hơn đến các ứng dụng thực tế như giám sát giao thông hoặc an toàn lao động.